VELTECH-AMS-MINI

Project report submitted
in partial fulfillment of the requirement
for award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

By

[STUDENT NAME 1] ([REGISTER NO. 1])
[STUDENT NAME 2] ([REGISTER NO. 2])
[STUDENT NAME 3] ([REGISTER NO. 3])
[STUDENT NAME 4] ([REGISTER NO. 4])

10211CS212 - WEB AND MOBILE APPLICATION DEVELOPMENT

SUMMER 2026-2027

Under the guidance of
[FACULTY NAME], [QUALIFICATION]
ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University Estd u/s 3 of UGC Act, 1956)
Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA

[MONTH, YEAR]

CERTIFICATE

It is certified that the work contained in the project report titled "VELTECH-AMS-MINI" by [STUDENT NAME 1] ([REGISTER NO. 1]), [STUDENT NAME 2] ([REGISTER NO. 2]), [STUDENT NAME 3] ([REGISTER NO. 3]) and [STUDENT NAME 4] ([REGISTER NO. 4]) has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor
[FACULTY NAME]
ASSISTANT PROFESSOR
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science & Technology
[MONTH, YEAR]

Signature of Head/Assistant Head
of the Department

Dr. M. Kavitha / Dr. T. Kujani
Professor & Head / Associate Professor & Assistant Head
Computer Science & Engineering
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
[MONTH, YEAR]

Signature of the Dean

Dr. S. P. Chokkalingam
Professor & Dean
School of Computing
Vel Tech Rangarajan Dr. Sagunthala R&D
Institute of Science and Technology
[MONTH, YEAR]

DECLARATION

We declare that this written submission represents our ideas in our own words and, where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea, data, fact or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from sources which have not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
[STUDENT NAME 1]
Date: / /

(Signature)
[STUDENT NAME 2]
Date: / /

(Signature)
[STUDENT NAME 3]
Date: / /

(Signature)
[STUDENT NAME 4]
Date: / /

APPROVAL SHEET

This project report entitled “VELTECH-AMS-MINI” by [STUDENT NAME 1] ([REGISTER NO. 1]), [STUDENT NAME 2] ([REGISTER NO. 2]), [STUDENT NAME 3] ([REGISTER NO. 3]) and [STUDENT NAME 4] ([REGISTER NO. 4]) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners **Handling faculty**

[FACULTY NAME], [QUALIFICATION]

Date: / /

Place:

ABSTRACT

veltech-ams-mini is a unified student web application that brings together academic attendance and campus engagement. It combines the attendance functions of Unieve-Attandance-AI with the events and community functions of Unieven-AI into one responsive interface for Vel Tech students. The application presents student identity, current and upcoming classes, subject attendance, academic reports, registered courses and earned credits alongside events, reels, posts, connections and faculty-created forms.

The system uses a React/Vite interface, a Node.js/Express campus service and a private Python/-FastAPI academic service. The academic adapter reads supported authenticated AMS pages while retaining their report headings, semester divisions and source timestamps. University records remain separate from editable social profiles. Overall results use an interactive cumulative CGPA line graph, clearly labelled published or calculated values, and semester-level PDF and Excel exports. Persistent campus sessions, incremental synchronization, bounded login admission and visibility-aware polling improve continuity and reduce unnecessary requests.

The project verification record reports 607 passing automated tests, supplemented by browser, responsive-layout and Docker integration checks. Earlier authorized AMS reads verified profile, course, attendance, timetable, results and earned-credit structures for one account. Unavailable source sections are identified explicitly. The project provides an extensible student workspace while preserving the boundary between university-owned records and campus-managed activities.

Keywords: Academic Management System, attendance, campus events, student community, React, FastAPI, source-aware synchronization, progressive web application.

LIST OF FIGURES

3.1 **Project architecture** 5
3.2 **Academic and campus data flow** 6

9.1 **Student Home workspace with demonstration data** 16
9.2 **Results line graph and overall cards.** The 8.40 CGPA and 84% shown are labelled demonstration estimates. 17

LIST OF TABLES

2.1 Software specification 4

4.1 Representative verification outcomes 9

LIST OF ACRONYMS AND ABBREVIATIONS

AMS	Academic Management System
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CSRF	Cross-Site Request Forgery
DOM	Document Object Model
GPA	Grade Point Average
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
OTP	One-Time Password
PDF	Portable Document Format
PWA	Progressive Web Application
QR	Quick Response code
REST	Representational State Transfer
TLS	Transport Layer Security
UI	User Interface
XLSX	Office Open XML spreadsheet format

TABLE OF CONTENTS

Page.No

ABSTRACT **iv**

LIST OF FIGURES **v**

LIST OF TABLES **vi**

LIST OF ACRONYMS AND ABBREVIATIONS **vii**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Aim of the project 1

1.3 Project Domain 1

1.4 Scope of the Project 2

1.5 Methodology 2

2 REQUIREMENT SPECIFICATION **3**

2.1 User characteristics 3

2.2 Dependencies 3

2.3 Hardware specification 3

2.4 Software specification 4

3 WEBSITE DESIGN **5**

3.1 Sitemap 5

3.2 Design Phase 5

3.2.1 Data Flow Diagram 6

3.3 Front End and Back End Design 6

3.3.1 Home Page 6

3.3.2 Campus Login and Session Restoration 6

3.3.3 Form Validation 7

3.3.4 Parsing AMS Webpages with Python and the DOM 7

3.3.5 Creation of Webserver Using Node.js 7

3.3.6 Design of the Three-Tier Application with MongoDB 7

3.3.7 Design of Reactive Forms in React 7
3.3.8 Routing and Navigation in React 7
3.3.9 Creation of Microservices 8
3.3.10 Deployment of Microservices 8

4 TESTING **9**
4.1 Testing 9
4.1.1 Test Result 9
4.1.2 Test Bugs 10

5 WEBSITE LAUNCH **11**

6 RESULTS AND DISCUSSIONS **12**
6.1 Website performance 12
6.2 Security 12
6.3 Responsiveness and mobile-friendliness 13

7 CONCLUSION AND FUTURE ENHANCEMENTS **14**
7.1 Conclusion 14
7.2 Future Enhancements 14

8 SOURCE CODE **15**

9 SCREENSHOTS **16**

References **18**

Chapter 1

INTRODUCTION

1.1 Introduction

veltech-ams-mini combines the Unieven-AI event/community project and the Unieve-Attandance-AI attendance project into one student web application [1, 2]. It is an independent student project for Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, Chennai. The combined application presents supported university AMS records [3], daily classes, campus activities and student interaction through a common responsive interface [4].

Students can review attendance and academic progress without leaving the campus workspace. Events, posts, reels, connections and faculty forms complement the academic functions. Official college identity remains separate from editable social profiles. Source timestamps and unavailable states help distinguish current, retained and missing information.

1.2 Aim of the project

The aim is to provide a clear, unified student workspace with the following objectives:

1. Present supported AMS student details, class timing, attendance, results and credits.
2. Combine event registration and community participation with scoped staff management.
3. Improve session continuity, synchronization feedback and phone navigation.
4. Provide repeatable Windows/Docker setup and permitted PDF/Excel exports.

1.3 Project Domain

The project belongs to educational information systems, campus event management and student community applications. React/Vite provides the interface, Node.js/Express manages campus workflows, and a private Python/FastAPI service reads supported AMS reports. MongoDB stores campus records and academic snapshots.

1.4 Scope of the Project

The academic scope includes source student facts, subject attendance and calendars, Day/Week/Full timetable, registered courses, results, assessments and earned credits. Fees, receipts, documents and examinations depend on supported source records. Campus functions include free/paid event workflows, tickets, posts/reels, comments, saves, connections, faculty forms, notifications and settings.

Official course submission remains an AMS portal handoff. Paid tickets require configured, server-verified payment confirmation. Missing placement or university data remains developing/unavailable. No supported independent Home of VelTech private API is configured; the retired parent portal is unused.

1.5 Methodology

Development consolidates the two repositories, separates campus and academic services, maps inspected source tables, and builds reusable responsive views. Validation combines automated tests, fictional API/browser workflows and earlier authorized read-only AMS observations. Successful sections appear incrementally; failed sections retain accepted records with their original timestamps. Bounded work queues and visibility-aware polling reduce redundant requests.

Chapter 2

REQUIREMENT SPECIFICATION

2.1 User characteristics

Students need readable phone navigation, private academic access, event registration and contextual social actions. Faculty need scoped forms, evidence review, publications and response exports. Organizers manage their assigned club or department; college event managers manage college-wide events. Administrators assign roles, publish notices, manage calendar/room policies and respond to app feedback.

Permissions come from persisted server assignments. Editing a social department, name or email cannot grant staff access. Academic identity is read-only; social names, bio and links use separate editable fields.

2.2 Dependencies

Academic retrieval requires reachable AMS pages and a valid upstream session. University verification must be completed when AMS requests it. Atlas requires a database user, database permissions and network access. Real recovery email, paid events and push delivery require configured providers. Camera, location and notification functions depend on browser permissions and deployed HTTPS. Free storage quotas are finite [5].

2.3 Hardware specification

The repository does not establish a measured minimum CPU/RAM requirement. Development needs a Windows computer capable of running Node.js and Docker Desktop with Linux containers. Deployment needs persistent disk, network access and a TLS reverse proxy. Students need a current desktop/phone browser; camera and location are optional workflow-specific capabilities. Host capacity must be established through deployment-specific testing.

2.4 Software specification

Table 2.1 summarizes the technologies used in the system.

Table 2.1: Software specification

Technology	**Use**
HTML, CSS, JavaScript	Accessible structure, presentation and interaction.
React, Vite	Component interface and production/PWA build [6, 7].
Tailwind CSS, Bootstrap	Utilities/grid with project-owned components.
React Router, Axios	Navigation and HTTP communication.
Node.js, Express	Campus REST API, sessions, events and community.
Python, FastAPI	Private academic integration service [8].
HTTPX, Requests, APScheduler	Portal requests and scheduled/background tasks.
PyMongo, Mongoose, MongoDB	Academic and campus persistence.
JWT, HttpOnly cookies	Private API authentication and browser sessions.
Docker Compose, PowerShell	Windows service isolation and setup [9].
PWA, IndexedDB	Installable shell and explicit local download copies.

Chapter 3

WEBSITE DESIGN

3.1 Sitemap

The shared shell links Home, Attendance, Timetable, Academics, Course registration, Events, Placements, Community, Connections, Feedback & forms, Academic calendar, Student details and Settings. Management appears according to staff permissions. Desktop sidebar navigation becomes bottom navigation with More on phones.

Figure 3.1 shows the delivered logical architecture. The browser contacts the campus API; its owner-bound bridge reaches the private academic API. The diagram describes this project, not the university's internal infrastructure.

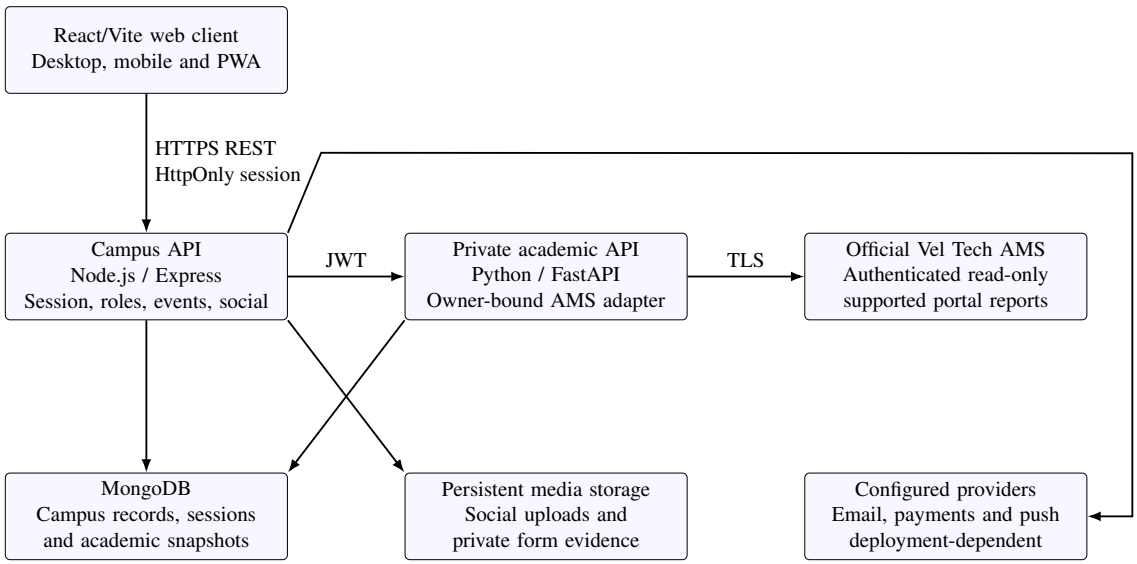


Figure 3.1: **Project architecture**

3.2 Design Phase

Presentation, campus business logic and academic integration have separate responsibilities. Source identity, editable social identity and authorization are distinct. Results retain semester, report and examination-session divisions; earned-credit course rows remain separate from category totals.

3.2.1 Data Flow Diagram

Identity/scope checks precede private reads and campus writes. Supported academic snapshots retain owner and provenance; social media and private form evidence have separate visibility rules.

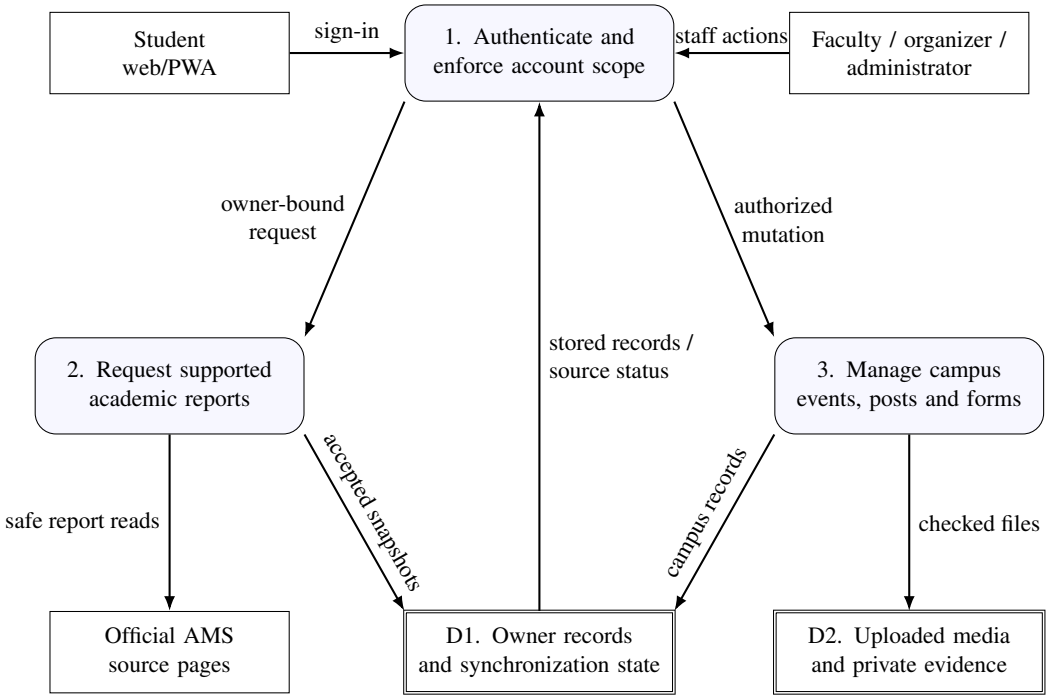


Figure 3.2: **Academic and campus data flow**

3.3 Front End and Back End Design

3.3.1 Home Page

The first content panel shows the current/next class using India time and actual source intervals. A compact student panel shows available official facts and portrait. Overall/subject attendance and notices follow. Only dated supplied marks determine present/absent colors; missing marks stay neutral.

3.3.2 Campus Login and Session Restoration

The entry screen accepts AMS credentials and any university-required challenge. Before rendering login, the client checks for a valid campus session. Temporary failures offer retry; concurrent restores coalesce and stale account responses are discarded. The rolling 30-day campus session is separate from the 30-minute process-local AMS session, which can require renewed verification after expiry/restart.

3.3.3 Form Validation

Client/server checks cover required fields, lengths, identifiers, ownership and role scope. Faculty forms support typed fields, deadlines, private live-camera proof and optional location conditions. One-use challenges reject replay. GPS/camera evidence supports review but cannot guarantee physical presence. Scoped staff can export formula-safe responses.

3.3.4 Parsing AMS Webpages with Python and the DOM

Verified-TLS requests and structured HTML parsing read supported labelled reports and inspected attendance grids. The source Student Session denominator is not replaced by planned sessions. Unknown layouts remain unavailable; accepted snapshots retain their timestamp. The adapter does not submit university registrations, feedback or payments.

3.3.5 Creation of Webserver Using Node.js

The Express service exposes campus sessions, academic bridging, events, posts, forms, notifications and management. Request limits, ownership, CSRF and file checks protect relevant routes. Private API responses use no-store handling; the built frontend can be served by the campus container.

3.3.6 Design of the Three-Tier Application with MongoDB

React forms the presentation tier; campus/academic services form the application tier; MongoDB/-media storage form persistence. Users own revocable sessions and academic links. Events own registrations, posts own interactions, and staff forms own scoped responses. Official academic data is excluded from public social queries.

3.3.7 Design of Reactive Forms in React

Controlled fields provide validation, busy, success and retry states. Event tickets await verified server payment state. Comments stay in context, with editing limited to five minutes. Likes and saves update optimistically, then reconcile with authoritative server state or roll back on failure.

3.3.8 Routing and Navigation in React

Dedicated event, ticket and attendance routes preserve navigation context. Wide tables/charts scroll inside their panels. Reels use full-screen viewing; replies remain in the active content. Profiles include editable bio/links and saved/liked/archive views. Recommendations use heuristics, not a claimed trained Instagram model.

3.3.9 Creation of Microservices

The campus API owns social/event workflows; the private FastAPI service owns AMS integration. Report jobs share three active slots and one per owner. Login admission separately permits four active and eight queued attempts with a five-second deadline. Busy requests return HTTP 503/Retry-After; cancellation releases reservations.

3.3.10 Deployment of Microservices

Docker Compose provides private networking and persistent media. The delivered topology uses one host and one academic worker because university sessions are process-local. Windows uses PowerShell, Docker Desktop, Node.js and Atlas. HTTPS, secure cookies, restricted origins and provider configuration are required for public deployment.

Chapter 4

TESTING

4.1 Testing

Verification separates automated regression, fictional integration/browser checks and historical live-source observations. The project verification record reports 607 passing tests: 163 frontend, 253 campus API, 180 Python academic-service and 11 Android configuration/packaging checks, with no failures/skips [4]. The production build and Docker health checks passed. These are recorded project results, not a new university test performed for this report.

4.1.1 Test Result

Table 4.1 summarizes representative outcomes. Android checks do not establish a compiled/signed APK.

Table 4.1: Representative verification outcomes

Scenario	**Recorded outcome**
Session continuity	Refresh/session restoration, transient retry and explicit logout verified.
Academic results	Credit joins, published summaries, selected-scale estimates, filters, PDF/Excel and keyboard graph tested.
Polling	Zero hidden requests and zero overview reloads for unchanged sync state in focused checks.
Phone layout	Login, Events and About checked at 320/390/1440 pixels in light/dark themes.
Permissions and notifications	Owner/staff boundaries, room changes, feedback retry and alert deduplication verified.
Work admission	Configured login/report capacities and cancellation checked with synthetic owners.

Earlier authorized AMS reads verified 21 profile fields, five current courses, 173 result/assessment rows and 71 credit rows for one account. Earlier attendance evidence included 22 timetable entries, five attendance summaries and 284 dated marks. No supported fee ledger or Course Material document rows were available for that account. These observations do not establish every account's current data.

4.1.2 Test Bugs

Regression coverage checks blank source-credit joins, ignored published summaries, refresh showing login prematurely, stale account responses, redundant background polling, mobile event/menu collisions and repeated feedback/absence alerts. Regressions exercise the corrected behavior. Live Atlas/Windows operation, physical-device camera/GPS, background sound/push, real mail, merchant payments and production failover remain deployment checks.

Chapter 5

WEBSITE LAUNCH

The Windows development path uses Node.js, Docker Desktop with Linux containers and MongoDB Atlas. No separate Ubuntu distribution or local MongoDB installation is required. After extracting the project, create an Atlas database user, grant the required database access and allow the host's network address. Keep the connection URI private.

From PowerShell in the application folder:

```
cd "P:\Intenship\veltech-ams-mini"
powershell -NoProfile -ExecutionPolicy Bypass -File .\Setup.ps1
npm run setup
npm run dev
```

Listing:

Setup writes persistent private configuration and builds the services. The React development app opens at `http://localhost:5173`; the built Docker app is served at `http://localhost:4100`. Use `npmrundoctor` for checks, `Start.ps1` to start and `Stop.ps1` to stop while retaining persistent data.

Persistent storage retains `.private`, encryption keys, environment settings, database names and media volumes. Public deployment adds HTTPS, restricted origins, secure cookies, database access and backup procedures. Expose the campus service while keeping the academic service private. Configure email and merchant/webhook providers before relying on those workflows.

The web build is an installable PWA. Android packaging requires a real HTTPS `deployment Url` in `android/config.local.json`, Android SDK, private signing identity and Digital Asset Links. Packaging tools are delivered; no signed APK is claimed. A static preview demonstrates layout using fictional data and cannot retrieve private academic records.

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Website performance

The combined workspace brings academics, events and community into one interface. Results begin with an interactive cumulative CGPA line graph, a semester-GPA option and numeric overall cards. Overall values remain independent of detail filters. Source report, semester, session, subject and individual assessment filters keep university divisions visible.

Published AMS values take precedence. When explicit eligible credits c_i and numeric grade points g_i are complete, the credit-weighted calculation is

$$\mathrm{GPA} = \frac{\sum_i c_i g_i}{\sum_i c_i}, \qquad \sum_i c_i > 0. \tag{6.1}$$

Cumulative calculations span eligible available semesters. Missing credits, conflicting attempts or unsupported grades prevent a complete result. Source course weights are not replaced by category totals. Where only letter grades exist, the selected visible scale produces a labelled estimate; the requested default percentage estimate is $\mathrm{CGPA} \times 10$, with an editable factor. It is not a universal official conversion.

Attendance uses recorded attended/total classes and a separate planning calculator. For attended P, total T and target fraction $0 < t < 1$, the allowed additional absences are $\max(0, \lfloor P/t - T \rfloor)$; consecutive attended classes needed are $\max(0, \lceil (tT - P)/(1 - t) \rceil)$. These estimates do not change official records.

A historical local Docker benchmark used 24 fictional accounts, eight concurrent flows and 144 requests: zero failures in 5.0 seconds, with campus-login p50/p95 of 591.0/1019.9 ms. It generated no AMS traffic and is not a production capacity claim. Incremental sync, bounded admission and hidden-page polling suppression reduce unnecessary work.

6.2 Security

Campus sessions use persisted token hashes, HttpOnly cookies, expiry and logout revocation. CSRF and owner/role checks protect mutations. Academic bridge tokens use AES-256-GCM encryption

and private API authentication. Official identity remains separate from social edits. One-use evidence/ticket controls reduce replay; GPS alone cannot prove attendance [10].

Private academic responses are excluded from the offline shell cache. Downloads stores explicit account-scoped browser copies within finite limits: 100 files/50 MB, at most 15 MB per file, subject to browser quota. It cannot read another website's downloads. No credentials or real student records appear in this report's figures/source.

6.3 Responsiveness and mobile-friendliness

Navy/blue/gold styling, phone bottom navigation, wrapping content and panel-contained scrolling support narrow screens. Subject attendance precedes calendars; detail routes and date inspection provide context. Full-screen reels, in-context comments and optimistic social actions preserve continuity. The recorded responsive checks support these tested surfaces; physical-device installation and background notification behavior still require device validation.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

veltech-ams-mini consolidates the two existing projects into one academic and campus workspace. Source-aware attendance, timetable, results and credits share navigation with events, social interaction, faculty forms and notifications. Official records remain separate from editable campus content.

The application provides persistent session restoration, incremental synchronization, result calculations and mobile readability. Its recorded automated/browser/Docker checks support the delivered behavior. Missing source data is identified rather than invented; university registration writes and external provider operations retain their documented boundaries.

7.2 Future Enhancements

Priorities include a documented university API agreement, more authorized source-layout fixtures, deployment-specific load/failure testing and backup recovery. Multi-host academic replication needs an explicit university-session strategy and shared media design. Real email, merchant and physical-phone workflows need operator validation.

A signed Android package can follow once HTTPS hosting and signing inputs are configured. Accessibility and moderation controls can expand further. Trained recommendations should be considered only with consented data and measured evaluation; the current community feed uses heuristics. Placement/source modules should grow only as reliable authorized records become available.

Chapter 8

SOURCE CODE

The selected excerpt below is copied from the current campus service. It hashes session tokens, checks a configured webhook signature and protects academic link tokens with authenticated encryption. Keys come from private server configuration; no secret value is reproduced.

```
import crypto from 'node:crypto';
export const hashToken = token => crypto.createHash('sha256').update(String(token)).digest('hex');
export const newToken = () => crypto.randomBytes(32).toString('hex');
export function checkWebhook(rawBody,signature,secret) {
  if (!Buffer.isBuffer(rawBody) || !secret || typeof signature!=='string' || !/^[a-f0-9]{64}$/i.test(
      signature)) return false;
  const expected=crypto.createHmac('sha256',secret).update(rawBody).digest();
  return crypto.timingSafeEqual(expected,Buffer.from(signature,'hex'));
}
function keyBuffer(key) {
  if (!/^[a-f0-9]{64}$/i.test(key || '')) throw new Error('CAMPUS_LINK_KEY must be a persistent 32-
      byte hex key');
  return Buffer.from(key,'hex');
}
export function encryptToken(token,key) {
  const iv=crypto.randomBytes(12),cipher=crypto.createCipheriv('aes-256-gcm',keyBuffer(key),iv);
  const body=Buffer.concat([cipher.update(token,'utf8'),cipher.final()]);
  return Buffer.concat([iv,cipher.getAuthTag(),body]).toString('base64');
}
export function decryptToken(sealed,key) {
  const bytes=Buffer.from(sealed,'base64');
  const cipher=crypto.createDecipheriv('aes-256-gcm',keyBuffer(key),bytes.subarray(0,12));
  cipher.setAuthTag(bytes.subarray(12,28));
  return Buffer.concat([cipher.update(bytes.subarray(28)),cipher.final()]).toString('utf8');
}
export function fail(status,message) { const error=new Error(message);error.status=status;throw
    error; }
export function httpUrl(value) {
```

Listing 8.1: Campus token and signature security helpers

The full application separates `frontend/`, `services/campus/` and `services/academic/`. Docker, setup and operator instructions are kept in `docker/`, `scripts/` and `docs/`. These excerpts illustrate selected functions; the complete application is maintained separately.

Chapter 9

SCREENSHOTS

These screenshots show the implemented interface using fictional demonstration records. Names, attendance and grades are sample data, not actual student records.

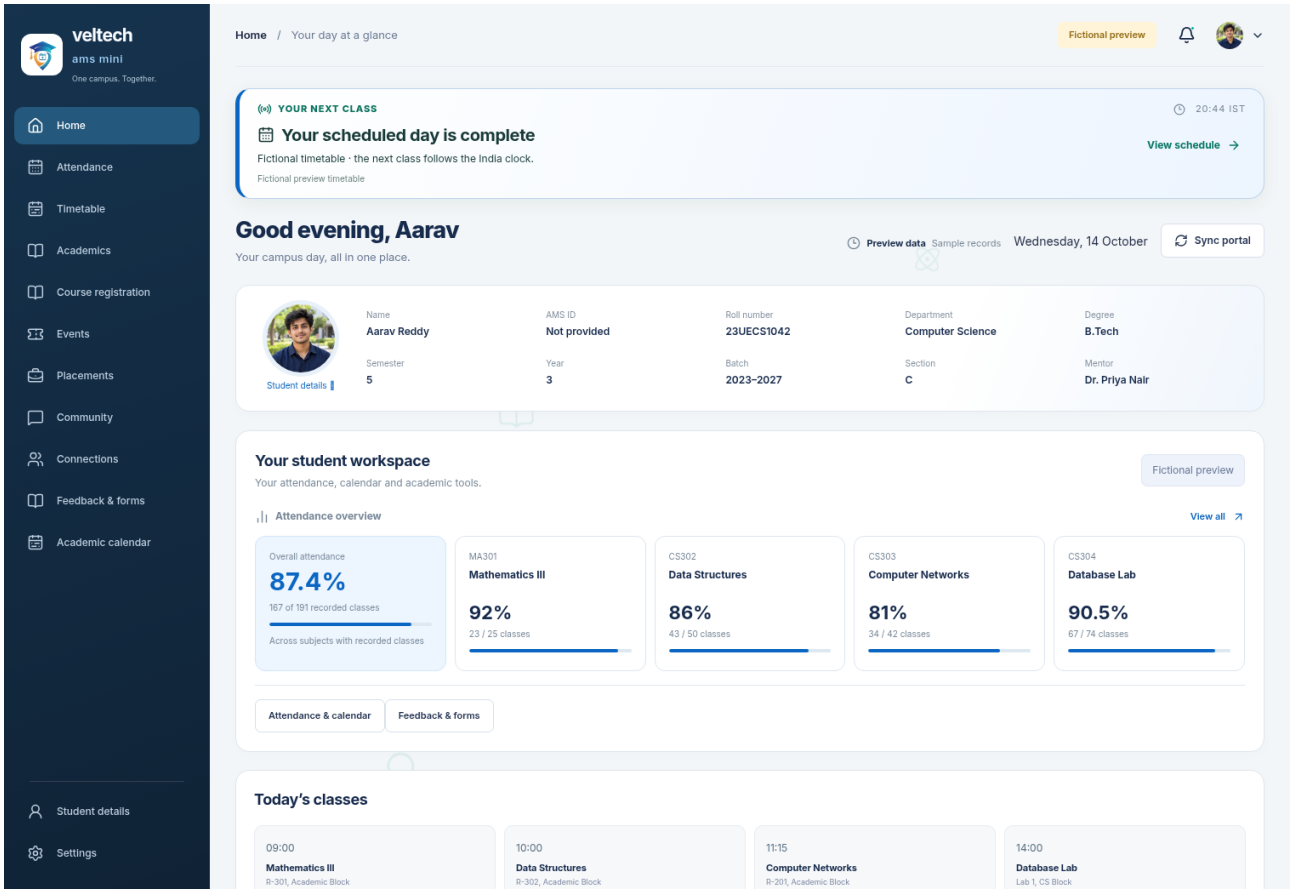


Figure 9.1: **Student Home workspace with demonstration data**

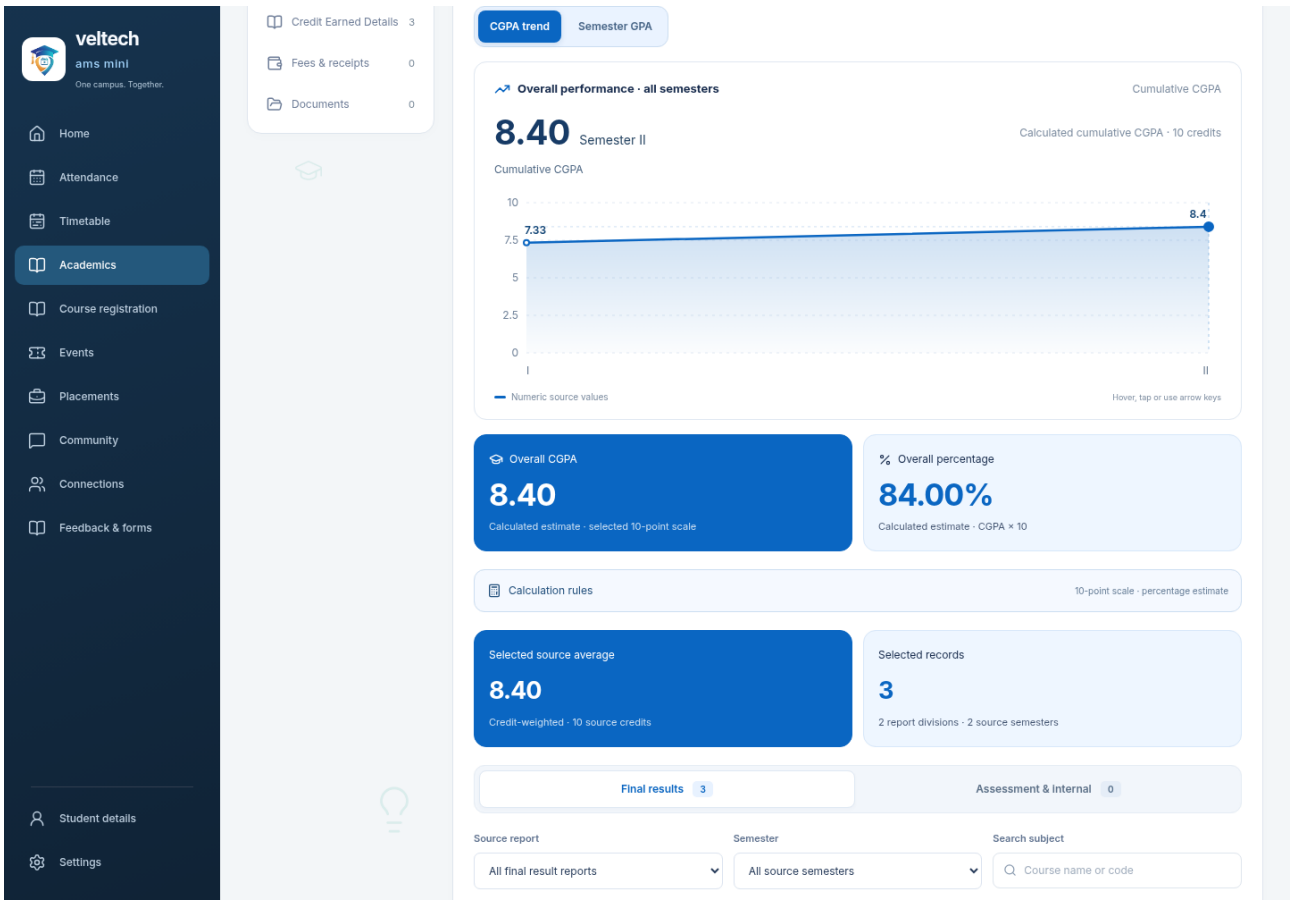


Figure 9.2: **Results line graph and overall cards.** The 8.40 CGPA and 84% shown are labelled demonstration estimates.

References

[1] mprem27. Unieven-AI. Project source repository. [Online]. Available: https://github.com/mprem27/Unieven-AI

[2] ——. Unieve-Attandance-AI. Project source repository. [Online]. Available: https://github.com/mprem27/Unieve-Attandance-AI

[3] Vel Tech. Academic Management System. [Online]. Available: https://ams.veltech.edu.in/Default.aspx

[4] VEL STUDENTS, *veltech-ams-mini: Source Code and Project Documentation*, Oct. 2026, README, feature, implementation, technology and verification records. [Online]. Available: https://github.com/vtu26418-design/Clinical/tree/e3d50e627dcbe15fc316bc1fe3a2c01570a7c08a

[5] MongoDB. Atlas free-cluster limitations. [Online]. Available: https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/

[6] React contributors. React documentation. [Online]. Available: https://react.dev/

[7] Vite contributors. Vite guide. [Online]. Available: https://vite.dev/guide/

[8] FastAPI contributors. FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com/

[9] Docker. Docker Compose documentation. [Online]. Available: https://docs.docker.com/compose/

[10] OWASP Foundation. Session management cheat sheet. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html